

 Práctica Evaluativa –Parcial I

Asignatura: Programación I

Tema: Estructuras Repetitivas con Listas

Caso 10 – Hotel – Habitaciones y estado (0=libre / 1=ocupada)

Enunciado / Descripción

Un hotel desea un sistema para controlar el estado de sus habitaciones (libre u ocupada). Se pide construir un menú que utilice listas paralelas: una lista habitaciones[] para almacenar los números de las habitaciones (e.g., 101, 102, 201) y otra lista estados[] para almacenar el estado de cada habitación, donde 0 representa "libre" (desocupada) y 1 representa "ocupada". Ambas listas deben compartir el índice, de manera que el estado en estados[i] corresponda a la habitación en habitaciones[i]. El programa debe usar un bucle while para presentar un menú al usuario, permitiéndole realizar diferentes operaciones hasta que elija la opción "Salir".

Ejemplo:

- habitaciones[] = [101, 102, 201]
- estados[] = [0, 1, 0]

En este ejemplo:

- La habitación 101 está libre (0).
- La habitación 102 está ocupada (1).
- La habitación 201 está libre (0).

Desafío: Implementar el sistema de gestión de habitaciones y estados utilizando listas paralelas y un menú interactivo.

Menú:

1. **Ingresar números de habitación:** (Registrar las habitaciones del hotel)
 - Permite al usuario ingresar los números de las habitaciones del hotel.
 - Ejemplo: El usuario ingresa "103", "104", "202".
2. **Ingresar estados (0/1) paralelos:** (Definir el estado inicial de cada habitación)
 - Permite al usuario ingresar el estado inicial de cada habitación, utilizando 0 para "libre" y 1 para "ocupada". Debe corresponder al orden de las habitaciones ingresadas previamente.
 - Ejemplo: Si las habitaciones son "103", "104" y "202", el usuario podría ingresar "0" para 103, "1" para 104 y "0" para 202.

3. **Mostrar estado general:** (Mostrar el estado de todas las habitaciones)
 - Muestra una lista de todas las habitaciones y su estado correspondiente (libre u ocupada).
 - Ejemplo de salida:
 - "Habitación 101: Libre"
 - "Habitación 102: Ocupada"
 - "Habitación 201: Libre"
 - "Habitación 103: Libre"
 - "Habitación 104: Ocupada"
 - "Habitación 202: Libre"
4. **Consultar estado de una habitación:** (Verificar el estado de una habitación específica)
 - Permite al usuario ingresar el número de una habitación y ver su estado actual (libre u ocupada).
 - Ejemplo: El usuario ingresa "102" y el programa muestra "Habitación 102: Ocupada".
5. **Listar ocupadas o libres (según lo que se pida):** (Mostrar una lista de habitaciones según su estado)
 - Permite al usuario elegir si quiere ver la lista de habitaciones ocupadas o la lista de habitaciones libres.
6. **Agregar habitación:** (Añadir una nueva habitación al sistema)
 - Permite al usuario agregar una nueva habitación al sistema, asignándole un estado inicial (libre u ocupada).
7. **Cambiar estado:** (Cambiar el estado de una habitación)
 - Permite al usuario cambiar el estado de una habitación (de libre a ocupada o de ocupada a libre). Debe solicitar el número de habitación y el nuevo estado deseado.
8. **Salir:** (Terminar el programa)
 - Termina la ejecución del programa.



Entregables

El estudiante deberá **subir el archivo del programa en lenguaje Python** a la plataforma

institucional. NO SUBIR UN REPOSITORIO DE GITHUB. **SOLO SUBIR EL ARCHIVO.PY**

El código debe cumplir con:

- Todas las funcionalidades solicitadas reflejadas en el menú.
- Buena ejecución sin errores.
- Nomenclatura clara en el nombre de las variables.
- Legibilidad general y buenas prácticas de codificación.

Rúbrica de Evaluación

Rúbrica de Evaluación Detallada para Gestión de Habitaciones de Hotel

Código	Criterio	Peso	Descripción Detallada
C1	Correctitud Funcional	50%	Este criterio evalúa si el programa cumple con las funciones principales del sistema de gestión de habitaciones de hotel. Esto incluye: • Agregar habitación: Permite añadir nuevas habitaciones al sistema con su estado inicial (libre u ocupada), asegurándose de que el número de habitación sea único. • Consultar estado: Permite verificar el estado (libre u ocupada) de una habitación específica a partir de su número. • Cambiar estado: Permite modificar el estado de una habitación existente (de libre a ocupada o viceversa). • Ver libres/ocupadas: Permite listar las habitaciones que están actualmente libres u ocupadas, según la selección del usuario. • Ver todas: Permite mostrar una lista completa de todas las habitaciones y su estado actual. • Manejar casos extremos sin errores: El programa no debe fallar o comportarse de forma inesperada ante entradas no válidas o situaciones límite. Debe proporcionar mensajes de error adecuados.
C2	Cumplimiento de Restricciones	20%	Este criterio evalúa el cumplimiento de las restricciones impuestas en el diseño del programa. Esto incluye: • Listas paralelas: Se deben utilizar listas paralelas para almacenar los números de habitación y sus estados correspondientes. No se permite el uso de otras estructuras de datos como diccionarios o clases. • Conservar habitaciones aunque estado cambie: El sistema debe mantener el registro de una habitación incluso si su estado cambia de libre a ocupada o viceversa. No se deben eliminar habitaciones del

Código	Criterio	Peso	Descripción Detallada
			sistema. • Sin clases/diccionarios: El uso de clases o diccionarios está prohibido y resultará en una penalización. Se busca evaluar la capacidad de resolver el problema utilizando listas. • Sincronía entre listas: El programa debe asegurar que la sincronización entre las listas se mantenga en todo momento. El estado en la posición i debe corresponder a la habitación en la posición i de la otra lista.
C3	Interacción y Validación	10%	Este criterio evalúa la calidad de la interacción con el usuario y las validaciones implementadas. Esto incluye: • Número de habitación no vacío: El programa debe verificar que el número de habitación ingresado por el usuario no esté vacío. • Estado inicial válido (0 o 1): El programa debe validar que el estado inicial ingresado para una habitación sea un valor válido (0 para libre, 1 para ocupada). • Cambio de estado solo si existe la habitación: Al intentar cambiar el estado de una habitación, el programa debe verificar que la habitación exista en el sistema. • Menú persistente: El menú principal del programa debe permanecer visible y funcional hasta que el usuario seleccione la opción de salir. • Opciones inválidas con mensaje claro: El programa debe manejar las opciones inválidas del menú con gracia, mostrando un mensaje claro y comprensible al usuario.
C4	Estructura y Legibilidad	10%	Este criterio evalúa la calidad del código en términos de estructura, legibilidad y estilo. Esto incluye: • Variables descriptivas: Los nombres de las variables deben ser descriptivos y reflejar el propósito de la variable. • Mensajes claros: Los mensajes mostrados al usuario deben ser claros, concisos y fáciles de entender. • Estructura: El código debe estar bien organizado y estructurado, con una lógica clara y fácil de seguir. Se debe utilizar indentación consistente. • Evitar duplicación: El código debe evitar la duplicación innecesaria de código. Se deben utilizar funciones para reutilizar la lógica común.
C5	Casos de Prueba / Datos de	5%	Este criterio evalúa la variedad y cobertura de los casos de prueba utilizados para verificar la funcionalidad del

Código	Criterio	Peso	Descripción Detallada
	Ejemplo		programa. Esto incluye: - Habitaciones libres: Probar el sistema con habitaciones en estado libre. - Cambio de estado: Probar el cambio de estado de una habitación (de libre a ocupada y viceversa). - Consulta habitación inexistente: Probar la consulta de una habitación que no existe en el sistema. - Estado inválido: Probar el ingreso de un estado inválido para una habitación (ej., un número distinto de 0 o 1). - Etc.: Se deben incluir otros casos de prueba que cubran diferentes escenarios y situaciones límite.
C6	Gestión de Casos Borde	5%	Este criterio evalúa la capacidad del programa para manejar situaciones límite o casos especiales que pueden surgir. Esto incluye: - Estado inválido: Intentar ingresar un estado inválido para una habitación (ej., un número distinto de 0 o 1). - Habitación inexistente: Intentar realizar una operación (consultar, cambiar estado) en una habitación que no existe. - Duplicados: Intentar agregar una habitación con un número que ya existe en el sistema. - Número vacío: Intentar agregar una habitación con un número de habitación vacío.

Descriptorios globales por nivel

Excelente (90–100): Cumple todos los criterios sin fallas; maneja casos borde exhaustivamente; interacción robusta; código claro y consistente.

Muy Bueno (80–89): Cumple la mayoría; pequeños desajustes no críticos; validación adecuada; estilo mayormente claro.

Aprobado (60–79): Cumple los básicos; algunos fallos funcionales menores o validación incompleta; legibilidad aceptable.

Insuficiente (<60): Incumple criterios clave; errores que impiden el uso; omite restricciones solicitadas.

Penalizaciones y observaciones

- 30% si utiliza estructuras prohibidas (diccionarios, clases, etc.).
- 10% si no conserva ítems con cantidad 0 (elimina de una lista y no de la otra).
- 10% por ausencia total de validación de entradas.
- 5% por mensajes ininteligibles o menú que no persiste.

Notas

- Asignar puntajes parciales por criterio y calcular la nota final ponderada.

- Compartir la rúbrica con estudiantes antes de la evaluación.
- Mantener consistencia entre enunciados, resultados de aprendizaje y criterios de evaluación.

